



National Textile University

Department of Computer Science

Subject:

Operating System

Submitted to:

Nasir Mahmood

Submitted by:

Afia Maham

Reg number:

23-NTU-CS-1126

Lab no:

4 - hometask

Semester:

5th

Task01:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
typedef struct {
```

```
    int id;
```

```
    char *message;
```

```
} ThreadData;
```

```
void* printData(void* arg) {
```

```
    ThreadData* data = (ThreadData*)arg;
```

```
    printf("Thread %d says: %s\n", data->id, data->message);
```

```
    return NULL;
```

```
}
```

```
int main() {
```

```
    pthread_t threads[3];
```

```
    ThreadData data[3] = {
```

```
        {1, "Hello from thread 1!"},
```

```
        {2, "Greetings from thread 2!"},
```

```
        {3, "Hi there from thread 3!"}
```

```
    };
```

```
    for (int i = 0; i < 3; i++) {
```

```
        pthread_create(&threads[i], NULL, printData, &data[i]);
```

```
    }
```

```

for (int i = 0; i < 3; i++) {
    pthread_join(threads[i], NULL);
}

printf("All threads have finished.\n");
return 0;
}

```

The screenshot shows the Visual Studio Code editor with a C program named `task01.c` open. The program defines a `ThreadData` struct with `number` and `isPrime` fields, and a `printData` function. The `main` function creates three threads, each printing a message and then being joined. The terminal output shows the program being compiled and executed, with the expected output messages appearing in sequence.

```

lab04-hometask > C task01.c
11 void* printData(void* arg) {
15 }
16
17 int main() {
18     pthread_t threads[3];
19     ThreadData data[3] = {
20         {1, "Hello from thread 1!"},
21         {2, "Greetings from thread 2!"},

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
afiamaham@DESKTOP-190QPQH:~/Operating System$ gcc lab04-hometask/task01.c -o lab04-hometask/task01-out
afiamaham@DESKTOP-190QPQH:~/Operating System$ lab04-hometask/task01-out
Thread 1 says: Hello from thread 1!
Thread 2 says: Greetings from thread 2!
Thread 3 says: Hi there from thread 3!
All threads have finished.
afiamaham@DESKTOP-190QPQH:~/Operating System$

```

Task02:

```

#include <stdio.h>
#include <pthread.h>
#include <stdbool.h>

```

```

typedef struct {
    int number;
    bool isPrime;
} ThreadData;

```

```
void* checkPrime(void* arg) {  
    ThreadData* data = (ThreadData*)arg;  
    int n = data->number;  
  
    if (n <= 1) {  
        data->isPrime = false;  
        return NULL;  
    }  
  
    data->isPrime = true;  
    for (int i = 2; i * i <= n; i++) {  
        if (n % i == 0) {  
            data->isPrime = false;  
            break;  
        }  
    }  
  
    return NULL;  
}
```

```
int main() {  
    pthread_t thread;  
    ThreadData data;  
  
    printf("Enter a number: ");
```

```
scanf("%d", &data.number);
```

```
pthread_create(&thread, NULL, checkPrime, &data);
```

```
pthread_join(thread, NULL);
```

```
if (data.isPrime)
```

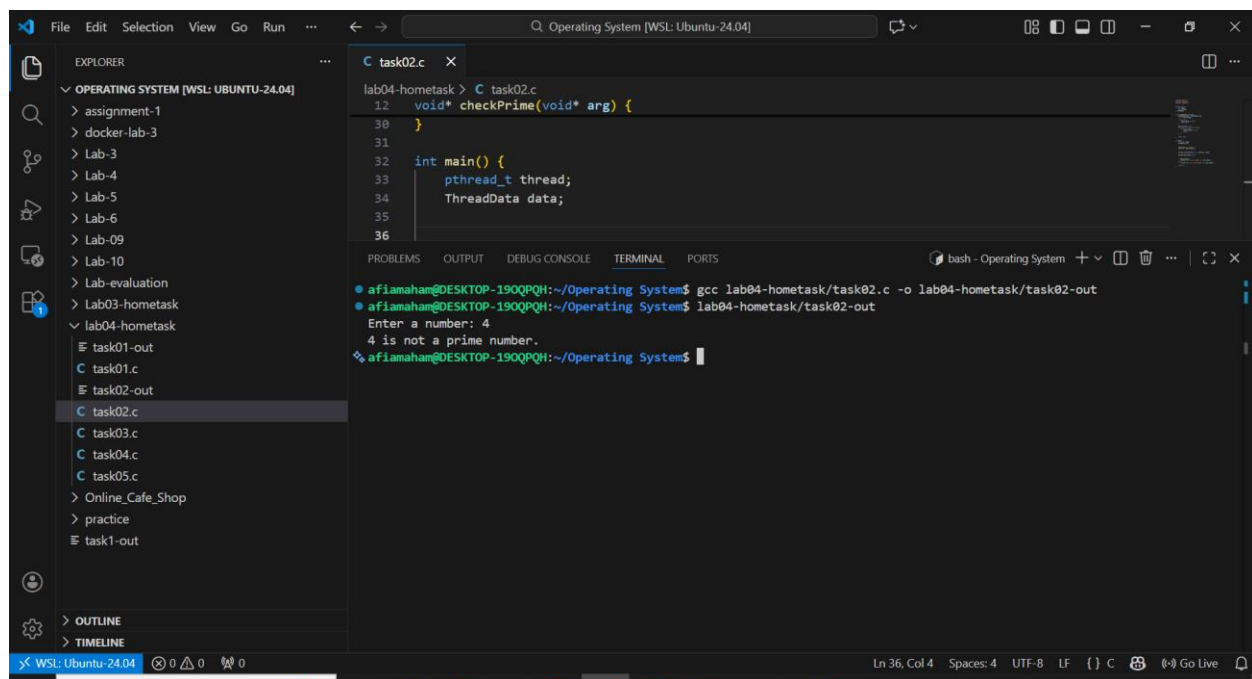
```
    printf("%d is a prime number.\n", data.number);
```

```
else
```

```
    printf("%d is not a prime number.\n", data.number);
```

```
return 0;
```

```
}
```



The screenshot shows the Visual Studio Code interface with a C program named `task02.c` open in the editor. The program defines a `checkPrime` function and a `main` function that creates a thread to check if a number is prime. The terminal output shows the compilation and execution of the program, with the input `4` resulting in the output `4 is not a prime number.`

```
lab04-hometask > C task02.c
12 void* checkPrime(void* arg) {
30 }
31
32 int main() {
33     pthread_t thread;
34     ThreadData data;
35
36
```

```
afiamaham@DESKTOP-190QPQH:~/Operating System$ gcc lab04-hometask/task02.c -o lab04-hometask/task02-out
afiamaham@DESKTOP-190QPQH:~/Operating System$ lab04-hometask/task02-out
Enter a number: 4
4 is not a prime number.
afiamaham@DESKTOP-190QPQH:~/Operating System$
```

Task03:

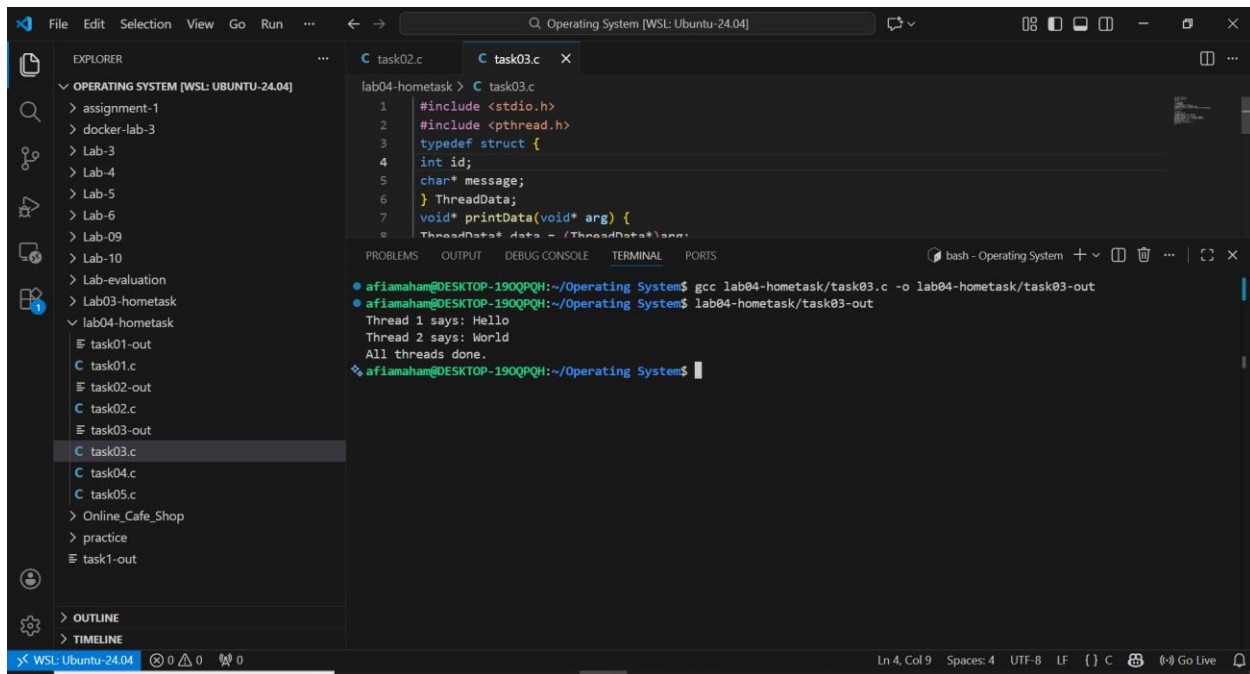
```
#include <stdio.h>

#include <pthread.h>

typedef struct {
    int id;
    char* message;
} ThreadData;

void* printData(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    printf("Thread %d says: %s\n", data->id, data->message);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    ThreadData data1 = {1, "Hello"};
    ThreadData data2 = {2, "World"};
    pthread_create(&t1, NULL, printData, &data1);
    pthread_create(&t2, NULL, printData, &data2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("All threads done.\n");
    return 0;
}
```



Task04:

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
void* worker_thread(void* arg) {
```

```
    int thread_num = *(int*)arg;
```

```
    printf("Thread %d: Starting work...\n", thread_num);
```

```
    sleep(1);
```

```
    printf("Thread %d: Work completed!\n", thread_num);
```

```
    return NULL;
```

```
}
```

```
int main() {
```

```
    pthread_t threads[5];
```

```
    int thread_args[5];
```

```
    for (int i = 0; i < 5; i++) {
```

```

thread_args[i] = i + 1;

printf("Main: Creating thread %d\n", i + 1);

pthread_create(&threads[i], NULL, worker_thread, &thread_args[i]);
}

for (int i = 0; i < 5; i++) {

pthread_join(threads[i], NULL);

printf("Main: Thread %d has finished\n", i + 1);
}

printf("All threads completed!\n");

return 0;
}

```

The screenshot shows a Visual Studio Code editor window with the following components:

- Explorer Panel:** Displays a file tree for the 'lab04-hometask' project, including files like 'task01.c', 'task02.c', 'task03.c', 'task04.c', and 'task05.c'.
- Editor Panel:** Shows the source code for 'task04.c'. The code defines a 'worker_thread' function that takes an argument, prints its ID, sleeps for 1 second, and prints 'Work completed!'. The main function creates 5 threads, each with an argument from 1 to 5, and joins them.
- Terminal Panel:** Shows the output of the program. It displays the sequence of thread creation, work completion, and thread joining messages, confirming that all 5 threads completed successfully.

```

lab04-hometask > C task04.c
3 #include <unistd.h>
4 void* worker_thread(void* arg) {
5     int thread_num = *(int*)arg;
6     printf("Thread %d: Starting work...\n", thread_num);
7     sleep(1);
8     printf("Thread %d: Work completed!\n", thread_num);
9     return NULL;
10 }

afiamaham@DESKTOP-1900PQH:~/Operating System$ lab04-hometask/task04-out
Main: Creating thread 2
Thread 1: Starting work...
Main: Creating thread 3
Thread 2: Starting work...
Main: Creating thread 4
Thread 3: Starting work...
Main: Creating thread 5
Thread 4: Starting work...
Thread 5: Starting work...
Thread 1: Work completed!
Thread 2: Work completed!
Thread 3: Work completed!
Main: Thread 1 has finished
Main: Thread 2 has finished
Main: Thread 3 has finished
Thread 4: Work completed!
Thread 5: Work completed!
Main: Thread 4 has finished
Main: Thread 5 has finished
All threads completed!
afiamaham@DESKTOP-1900PQH:~/Operating System$

```

Task05:

```

#include <stdio.h>

#include <pthread.h>

#include <stdlib.h>

```



```
void* calculate_sum(void* arg) {
    int n = *(int*)arg;
    int* result = malloc(sizeof(int)); // Allocate memory for result
    *result = 0;
    for (int i = 1; i <= n; i++) {
        *result += i;
    }
    printf("Thread calculated sum of 1 to %d = %d\n", n, *result);
    return (void*)result; // Return the result
}

int main() {
    pthread_t thread_id;
    int n = 100;
    void* sum;
    pthread_create(&thread_id, NULL, calculate_sum, &n);
    // Get the return value from thread
    pthread_join(thread_id, &sum);
    printf("Main received result: %d\n", *(int*)sum);
    free(sum); // Don't forget to free allocated memory
    return 0;
}
```

